

Data Engineering Technical Exercise

KAD Global Agency | Confidential

Role:	Data Engineer
Technology:	Python 3.10+ (required). For transformations: pandas, polars, or DuckDB SQL. SQLite is allowed as a persistence layer (see §4.1) but not as a transformation engine.
Estimated Time:	5–7 hours
Submission:	Source code (Git repository or zip) + README + run instructions + sample outputs
Send to:	muhammed.khan@truxel.ai, monica@kad.agency

1. Objective

Build a small, correct, idempotent data pipeline that ingests noisy IoT telemetry batches, produces a curated table, computes hourly aggregates and a billing summary, and emits a data-quality report.

We are evaluating **data modeling**, **idempotent ingestion**, **time-series correctness** (late arrivals, out-of-order, duplicates), **aggregation correctness**, **code structure**, and **clarity of documentation**. We are not evaluating UI, deployment, or infrastructure.

2. Scenario

An operator runs a fleet of IoT devices distributed across customer sites. Each device emits periodic readings of a small set of numeric metrics and a discrete state. Readings are buffered locally and pushed to a central ingest endpoint as **JSON batches**. The ingest endpoint persists each batch as a line of NDJSON in object storage.

The network between devices and the ingest endpoint is unreliable. As a result:

- The same batch may be delivered **more than once** (publisher retries on lost acknowledgements).
- When a device reconnects after an outage, it flushes its local buffer; the batches it sends contain readings with **timestamps in the past**.
- Within a single hour's file, batches from different devices are **interleaved** and not sorted by timestamp.

You are given 30 days of synthetic data for a fleet of twelve devices across three sites (roughly 320,000 readings total in around 17,000 batches), plus a per-device-type rate card. Your job is to produce a curated table of readings, an hourly aggregate table, a billing table, and a data-quality report.

3. Provided Data

The exercise ships with a `data/` directory:

```

data/
├── raw/readings/                                # NDJSON batch files
│   ├── 2026-04-01T00.ndjson
│   ├── 2026-04-01T01.ndjson
│   └── ...                                     # one file per UTC hour, 30 days
├── devices.csv                                # device metadata
├── rate_card.csv                             # billing rates
└── README.md                                # field semantics & row counts

```

3.1 Raw batches — `data/raw/readings/*.ndjson`

Each line of an NDJSON file is exactly one batch. A batch has this shape:

```

{
  "batch_id": "9f3e1c2a-4a8b-4c66-9a02-1d44e1b27b8f",
  "device_id": "dev-014",
  "site_id": "site-03",
  "device_type": "type_a",
  "ingested_at": "2026-04-12T14:33:02.184Z",
  "readings": [
    {"ts": "2026-04-12T14:32:10.000Z", "value": 78.34, "rate": 0.13, "state":
"ACTIVE"},
    {"ts": "2026-04-12T14:32:20.000Z", "value": 78.02, "rate": 0.12, "state":
"ACTIVE"},
    {"ts": "2026-04-12T14:32:30.000Z", "value": 78.70, "rate": 0.11, "state":
"ACTIVE"}
  ]
}

```

Batch fields:

Field	Type	Notes
batch_id	string (UUIDv4)	Unique per logical batch. Duplicates carry the same value.
device_id	string	Stable per device. Foreign key to <code>devices.csv</code> .
site_id	string	Site the device belongs to. Always consistent with <code>devices.csv</code> .
device_type	enum	One of <code>type_a</code> , <code>type_b</code> , <code>type_c</code> .
ingested_at	ISO-8601 UTC	When the ingest endpoint received the batch. Always present.
readings	array	Zero or more reading objects. An empty list <code>[]</code> denotes an idle-heartbeat batch.

Reading fields (inside `readings[]`):

Field	Type	Notes
ts	ISO-8601 UTC	Event timestamp. Always present.

value	float	A dimensionless instantaneous measurement. Always present. Aggregated as min/avg/max in <code>hourly_aggregates</code> (§4.3).
rate	float	A signed instantaneous rate (units / second). Always present. The quantity that integrates to billable <code>delivered_units</code> (§4.3).
state	enum	One of <code>ACTIVE</code> , <code>IDLE</code> , <code>FAULT</code> . Always present.
temperature	float	Optional. May be absent on readings from older-firmware devices. When present, it is a finite float. Carried through to <code>readings_curated</code> but not consumed by any downstream table.

Properties of the dataset — your pipeline must handle these:

- **Duplicates.** Around 5–10% of distinct batches appear more than once with the same `batch_id` . Across duplicate occurrences of a given `batch_id` , all fields except `ingested_at` are identical: each retry is received later, so its `ingested_at` reflects the time at which the endpoint received that particular delivery. Duplicates may appear in the same file or in different files. After dedup, each row in `readings_curated` carries the `ingested_at` and `source_file` of the kept occurrence.
- **Out-of-order & late.** A reading's `ts` may be hours before the file it lands in. Treat `ts` (event time) as the source of truth for bucketing.
- **Schema drift.** The `temperature` field is optional. No other field may be missing.
- **Empty batches.** A batch may have `"readings": []` . These contain no measurements but must still be deduplicated and counted.
- **Bad data.** A small fraction of readings violate one or more anomaly rules in §4.2.

Input guarantees — you may rely on these without writing defensive code:

- Every line of every NDJSON file is valid JSON conforming to the batch shape above. No malformed lines, no extraneous fields.
- All fields documented as "Always present" are present and non-null on every batch and every reading.
- `value` , `rate` , and `temperature` (when present) are finite floats — never `NaN` or `±Infinity` .
- All timestamps are ISO-8601 with explicit UTC offset (`Z` or `+00:00`).

3.2 Device metadata — `data/devices.csv`

Column	Type	Notes
device_id	string	Primary key.
site_id	string	Site the device belongs to.
device_type	enum	One of <code>type_a</code> , <code>type_b</code> , <code>type_c</code> .
expected_interval_s	int	Expected seconds between consecutive readings under normal operation (used for §4.2 rule R4).

commissioned_at	ISO-8601 UTC	Readings before this timestamp should not exist; if present, ignore them and log them in the DQ report.
-----------------	--------------	---

3.3 Rate card — data/rate_card.csv

Column	Type	Notes
device_type	enum	Primary key.
unit_price	float	Price per unit of delivered_units (see §4.3). Billing formula in §4.4.
currency	string (ISO-4217)	Same for all rows in the shipped fixture.

Expected totals for every output table, exact anomaly counts per rule, and the tolerances on float-summed metrics all live in data/README.md . Read it before you start — your pipeline's correctness is graded against it.

4. Functional Requirements

4.1 Ingestion

Read every NDJSON file under `data/raw/readings/` and produce a curated table named `readings_curated` with the following columns:

Column	Type	Notes
<code>batch_id</code>	<code>string</code>	From the batch.
<code>device_id</code>	<code>string</code>	From the batch.
<code>site_id</code>	<code>string</code>	From the batch.
<code>device_type</code>	<code>string</code>	From the batch.
<code>ts</code>	<code>timestamp (UTC)</code>	From the reading.
<code>value</code>	<code>double</code>	From the reading.
<code>rate</code>	<code>double</code>	From the reading.
<code>state</code>	<code>string</code>	From the reading.
<code>temperature</code>	<code>double, nullable</code>	From the reading; null when absent.
<code>ingested_at</code>	<code>timestamp (UTC)</code>	From the batch.
<code>source_file</code>	<code>string</code>	POSIX path of the NDJSON file relative to the <code>--data-dir</code> argument, using forward slashes (e.g. <code>raw/readings/2026-04-12T14.ndjson</code>). The dedup rule's lexicographic ordering relies on this exact form.

Deduplication rule. If the same `batch_id` appears more than once across all input, keep the occurrence with the smallest `(source_file, line_number)` tuple (lexicographic on `source_file`, then numeric on line number). Discard all other occurrences. This rule is deterministic and does not depend on file-read order. The `source_file` column in `readings_curated` is the file of the kept occurrence.

Empty batches (`readings == []`) contribute zero rows to `readings_curated` but still participate in deduplication and are counted in the DQ report.

Idempotency. Running the pipeline twice on the same input must produce identical row sets in `readings_curated`.

Canonical sort order for this table on persistence: `(device_id, ts, batch_id)` ascending.

Persistence. Write `readings_curated` to disk as Parquet, DuckDB, or SQLite (your choice). Defend the choice briefly in the README.

4.2 Data Quality — Anomaly Rules

All four rules operate over the rows of `readings_curated` — that is, after deduplication and after the pre-commissioning filter. Duplicates and pre-commissioning readings do not contribute to any anomaly row and do not seed any rolling window.

Detect exactly the following four anomaly types. Persist them as a table named `dq_anomalies` :

Column	Type	Notes
<code>rule_id</code>	string	One of <code>R1</code> , <code>R2</code> , <code>R3</code> , <code>R4</code> .
<code>device_id</code>	string	
<code>site_id</code>	string	From <code>devices.csv</code> .
<code>ts</code>	timestamp (UTC)	See rule definitions below.
<code>severity</code>	string	One of <code>error</code> , <code>warning</code> .
<code>batch_id</code>	string, nullable	The <code>batch_id</code> of the reading that triggered the anomaly (<code>R1</code> , <code>R2</code> , <code>R3</code>). <code>null</code> for <code>R4</code> (no specific reading).
<code>details</code>	string	Rule-specific payload, formatted as defined per rule. Provided for human inspection; only the row count per rule is evaluation-checked, not the exact text of <code>details</code> .

Canonical sort order for this table on persistence: `(rule_id, device_id, ts)` ascending.

Rule	Definition	Severity
<code>R1</code> – <code>value_out_of_range</code>	Emit one anomaly per reading where <code>value < 0</code> or <code>value > 1000</code> . Use the reading's <code>ts</code> . <code>details</code> = the offending <code>value</code> formatted as a string.	error
<code>R2</code> – <code>fault_state</code>	Emit one anomaly per reading where <code>state == "FAULT"</code> . Use the reading's <code>ts</code> . <code>details</code> = the literal string <code>"FAULT"</code> .	warning
<code>R3</code> – <code>value_spike</code>	Per device, sort kept readings by <code>(ts, batch_id)</code> ascending. For each reading at position <code>i ≥ 30</code> (zero-indexed), compute μ and σ over the immediately preceding 30 readings (positions <code>i-30</code> to <code>i-1</code> of the same device). Use population standard deviation (<code>ddof = 0</code>) and float64 arithmetic. Emit an anomaly when $\sigma > 0$ and $ value - \mu > 5 \sigma$. Use the reading's <code>ts</code> . <code>details</code> = the z-score $(value - \mu) / \sigma$ formatted as a string rounded to two decimal places (e.g. <code>"7.43"</code>).	warning

R4 – device_silence	Restrict to devices whose <code>expected_interval_s ≤ 300</code> . Let <code>device_max_ts</code> = the maximum <code>ts</code> across that device's rows in <code>readings_curated</code> . Define <code>first_hour</code> = the smallest UTC hour boundary $\geq$ <code>commissioned_at</code> , and <code>last_hour</code> = the hour boundary at or just before <code>device_max_ts</code> (i.e. <code>floor(device_max_ts, hour)</code>). For every UTC hour boundary <code>H</code> with <code>first_hour ≤ H ≤ last_hour</code> for which the device has zero rows in <code>readings_curated</code> with <code>ts ∈ [H, H+1h)</code> , emit one anomaly. Hours strictly after <code>device_max_ts</code> are not silence anomalies (the device's data window closes at <code>last_hour</code>). Use <code>ts = H</code> . <code>details</code> = ISO-8601 string of <code>H</code> with millisecond precision and <code>Z</code> suffix (e.g. <code>"2026-04-12T14:00:00.000Z"</code>). error
------------------------	--

Natural key. The natural key of `dq_anomalies` is `(rule_id, device_id, ts)`. Your pipeline must produce at most one row per natural key. If multiple readings of the same device share the same `ts` and trigger the same rule (rare, but possible after dedup), emit one row and use the `batch_id` and `details` from the reading with the lexicographically smallest `batch_id`. Two consecutive runs on the same input must produce identical row sets.

Devices with no kept readings. If a device has zero rows in `readings_curated` (e.g. only pre-commissioning readings, or none at all), R3 and R4 emit zero rows for that device.

Pre-commissioning readings. Any reading with `ts < commissioned_at` for that device must be excluded from `readings_curated` and counted under the key `pre_commissioning_dropped` in `dq_summary.json` (it is *not* a row in `dq_anomalies`).

4.3 Hourly Aggregates

Operates on the rows of `readings_curated` (i.e. post-dedup, post-pre-commissioning filter). Produce a table `hourly_aggregates` with one row per `(device_id, hour_utc)` for which at least one reading exists. Bucket by event time: `hour_utc = date_trunc('hour', ts)` in UTC.

Column	Type	Definition
device_id	string	
site_id	string	
device_type	string	
hour_utc	timestamp (UTC)	Hour boundary.
samples_count	int	Number of readings in the hour.
value_avg	double	Arithmetic mean of <code>value</code> .
value_min	double	
value_max	double	
delivered_units	double	Trapezoidal integral of <code>rate</code> over time, in units. See formula below.

active_seconds	int	Seconds for which the most recent observed <code>state</code> was <code>ACTIVE</code> . See definition below.
----------------	-----	---

Within-hour sort. Within an hour bucket, sort readings by `(ts, batch_id)` ascending. This tie-break makes the trapezoidal and active-seconds calculations deterministic when two readings share the same `ts` .

Trapezoidal integral. For each consecutive pair `(r_i, r_{i+1})` within the hour add $0.5 \times (\text{rate}_i + \text{rate}_{i+1}) \times (\text{ts}_{i+1} - \text{ts}_i)_{\text{seconds}}$ in float64. Single-reading hours contribute `delivered_units = 0` . `delivered_units` is signed and may be negative.

Active seconds. For each consecutive pair within the hour, if the earlier reading's `state == "ACTIVE"` , add `(ts_{i+1} - ts_i)_seconds` . The trailing segment (from the last reading to the end of the hour) and the leading segment (from the start of the hour to the first reading) contribute 0. Single-reading hours contribute `active_seconds = 0` . Sum in float seconds, then round half-to-even (banker's rounding, Python's built-in `round()`) to the nearest integer and clamp to `[0, 3600]` .

Anomalous readings are still included in the aggregates — flagging is the DQ layer's job, not the aggregator's. The expected totals in `data/README.md` are computed including anomalous readings.

Canonical sort order for this table on persistence: `(device_id, hour_utc)` ascending.

4.4 Hourly Billing

Derives from `hourly_aggregates` (which itself derives from `readings_curated`). Produce a table `hourly_billing` with one row per `(site_id, hour_utc, device_type)` with at least one device-hour:

Column	Type	Definition
site_id	string	
hour_utc	timestamp (UTC)	
device_type	string	
total_delivered_units	double	Sum of <code>delivered_units</code> across devices of this type at this site in this hour.
amount	double	<code>total_delivered_units × unit_price</code> from the rate card. May be negative when <code>total_delivered_units</code> is negative; do not clip.
currency	string	From the rate card.

Canonical sort order for this table on persistence: `(site_id, hour_utc, device_type)` ascending.

4.5 Data-Quality Summary

Print a JSON summary to stdout at the end of every run, and write the same content to `out/dq_summary.json` . The JSON object must contain exactly these keys with the types shown:

Key	Type	Definition
files_read	int	Count of files matched by the glob <code><data-dir>/raw/readings/*.ndjson</code> , regardless of content.
batches_total	int	Total batch occurrences across all input files (before dedup).
batches_unique	int	Distinct <code>batch_id</code> s after dedup.
batches_duplicate_discarded	int	<code>batches_total - batches_unique</code> .
empty_batches	int	Number of distinct <code>batch_id</code> s (after dedup) whose <code>readings</code> array is empty. These contribute 0 to <code>readings_total</code> .
readings_total	int	Sum of <code>len(readings)</code> over the deduped batches, before the pre-commissioning filter is applied.
pre_commissioning_dropped	int	Readings excluded because <code>ts < commissioned_at</code> for their device.
readings_kept	int	<code>readings_total - pre_commissioning_dropped</code> . Equals the row count of <code>readings_curated</code> .
anomalies_by_rule	object	<code>{"R1": int, "R2": int, "R3": int, "R4": int}</code> .
anomalies_by_severity	object	<code>{"error": int, "warning": int}</code> .
duration_seconds	float	Wall-clock duration of the run, in seconds. Implementation-specific; not evaluation-checked.

4.6 What you are *not* asked to do

- No streaming, no message broker, no API server. File-in / file-out only.
- No backfill mode, no partial-replay flag. A single command runs the full pipeline.
- No authentication, multi-tenancy, encryption, or deployment.
- No UI, dashboard, notebook, or charts.

5. Pipeline Entry Point

The pipeline must be runnable with a single command. The exact command and package name are your choice; document them in the README. An example shape:

```
python -m <your_package> run --data-dir ./data --out-dir ./out
```

After a successful run, `./out/` must contain:

- `readings_curated.parquet` (or your chosen format)
- `dq_anomalies.parquet`
- `hourly_aggregates.parquet`
- `hourly_billing.parquet`
- `dq_summary.json`

Pick **one** storage mode for all four tables — either four Parquet files (as listed above) or a single `warehouse.duckdb` / `warehouse.sqlite` file containing all four tables. Do not mix modes.

`dq_summary.json` is always a separate file regardless of storage choice.

Out-dir behaviour. Existing output files in `out-dir` are overwritten on each run. Do not append. `out-dir` is created if it does not exist.

6. Technical Expectations

- Python 3.10+. Pin dependencies (`requirements.txt` , `pyproject.toml` , or `uv.lock`).
- Use a real data engine (pandas, polars, or DuckDB SQL) for transformations. Avoid Python row-by-row loops on hot paths.
- Modular package layout. Suggested: `pipeline/ingest.py` , `pipeline/quality.py` , `pipeline/aggregate.py` , `pipeline/billing.py` , `pipeline/cli.py` . Other layouts are fine if defensible.
- Structured logging (one JSON object per line) at each stage boundary, including row counts in and out.
- Deterministic output. Two consecutive runs on the same input must produce identical row sets in each output table, in canonical sort order by the table's natural key. Byte-identical files are not required — we will compare contents, not file hashes.
- **Tests — at least three:**
 - Dedup on a tiny fixture containing a known duplicate `batch_id` .
 - Trapezoidal integration on a hand-calculable fixture.
 - At least one anomaly rule (your choice) on a fixture where the expected anomaly rows are written by hand.
- The full pipeline must run end-to-end against the shipped fixture in **under 5 minutes** on a modern laptop.

7. Documentation (Required)

Include a `README.md` that covers, concisely:

- **How to run** — setup, full-pipeline command, how to run tests.
- **Architecture** — a short paragraph describing the stages. A diagram is welcome but not required.
- **Storage choice** — Parquet vs DuckDB vs SQLite, why.
- **Schema choice** — wide vs long for `readings_curated`, why.
- **Idempotency & determinism** — how dedup is enforced; what makes a repeat run produce identical output.
- **Late & out-of-order data** — how event-time bucketing prevents reordering issues.
- **Trade-offs you took** due to the 5–7h budget — what you cut, what you would build next.

Aim for two to four pages. We value concision and honest reasoning over completeness.

8. Submission

- Source code as a Git repository (preferred) or zip.
- `README.md` per §7.
- A `Makefile` or equivalent with: `make setup`, `make run`, `make test`.
- Paste the stdout of a successful run (the §4.5 summary) into the README or a separate `sample_output.txt`.

9. Evaluation Criteria

Criteria	What we look for
Correctness	Output tables match the expected totals and anomaly counts in <code>data/README.md</code> , within the tolerances stated there (exact for most counts; small documented tolerances for R3 and the float-summed metrics).
Idempotency & determinism	Re-running the pipeline yields identical row sets in every output table. Dedup rule is correctly applied.
Time-series correctness	Event-time bucketing handles late and out-of-order data. Trapezoidal integration is implemented correctly.
Data modeling	Sensible table layouts, types, and natural keys. Defensible wide-vs-long choice.
Code structure	Clear separation of ingest, quality, aggregate, billing. Small, testable functions. No dead code.
Testing	The required tests exist, pass, and exercise meaningful logic (not just smoke).
Use of tools	A real data engine is used for transformations rather than row-by-row Python.
Documentation	README is concise, answers the required questions, and includes an honest trade-offs section.

Good luck. We look forward to reviewing your submission.

— *KAD Global Agency, Data & Platform Team*